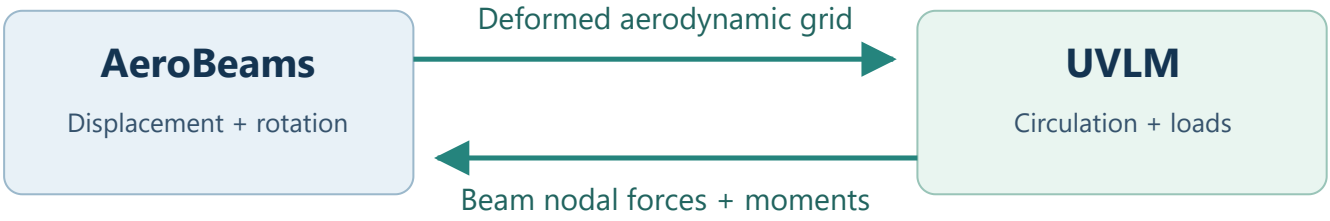
Understanding the UVLM + AeroBeams coupling

How the aerodynamic mesh, vortex loads and nonlinear beam work together, one time step at a time.

AeroBeams calculates how the flexible wing bends and twists. UVLM calculates the aerodynamic forces on that deformed wing. The coupling code exchanges geometry and loads until the two models agree closely enough.



> **One rule holds the calculation together:** repeated coupling trials solve the same next physical time. The accepted wake advances only once, after a trial is accepted.

Read in this order

2-3	Files, startup and model construction
4-6	Geometry transfer, UVLM loads and load transfer
7-9	Time stepping, coupling convergence and structural integration
10-12	Wake, outputs, static equilibrium and flutter estimates
13	File and function reference for reading the source

Based on the local walkthrough and source snapshot of 19 October 2026. Configuration values are examples from that snapshot. This guide explains the implementation; no new simulations were run to prepare it.

01 / WHERE EXECUTION STARTS

Start with the runner, then follow the coupling file

The runner chooses the case. The coupling function controls the interaction between the two solvers.

File in the Pazy example directory	Its job
`run_pazy_wing_uvlm.jl`	Set speed, incidence, mesh, time step, convergence controls and output options.
`PazyWingUVLMCoupling.jl`	Create the models, exchange geometry/loads, iterate coupling and advance time.
`PazyWingUVLMVisualization.jl`	Plot the response and optionally create deformation and wake animations.
`run_pazy_wing_uvlm_static.jl`	Run the separate static equilibrium procedure.
`run_pazy_stability_sweep.jl`	Repeat dynamic cases at several speeds and incidence angles.
`plot_pazy_flutter_boundaries.jl`	Plot manually entered flutter boundaries and comparison curves.

What happens when the dynamic runner executes?

1. **Load definitions.** The runner includes the coupling, visualization and moving-block damping files. `include` evaluates the Julia file.
2. **Choose the flight condition and controls.** These variables become keyword arguments to `run_pazy_wing_uvlm`.
3. **Run the coupled calculation.** The function returns a named tuple called `result`, with the problem objects, response histories and coupling diagnostics.
4. **Postprocess the result.** Selected branches create plots, estimate twist damping, or generate animations.

> **Two packages, one Julia process.** The coupling file imports `AeroBeams` and imports `WingPropellerUVLM` as `UVLM`. Geometry and load arrays pass between functions directly.

`WingPropellerUVLM.jl` loads many capabilities when imported. A function being available in the package does not mean this Pazy example calls it. The Pazy model contains one wing surface and no propeller.

Pazy example directory, relative to the repository root:
`lib/WingPropellerUVLM/examples/pazy_aerobeams_coupled/`

One beam model.
One aerodynamic model.

AeroBeams: flexible structure

`SampleModels.jl` supplies Pazy geometry, spanwise sectional stiffness and inertia.

The code builds a 15-element beam, clamps its root, adds nodal load boundary conditions, and creates a `DynamicProblem`.

UVLM: lifting surface + wake

The aerodynamic grid comes from beam geometry. The example snapshot uses 10 chordwise and 20 spanwise panels.

`create_UVLMDynamicProblem` allocates the accepted state, a trial workspace and an initially empty wake.

Structural setup

The root has zero displacement and rotation in all six components. Each of the 16 beam nodes also has six applied-load components:

```
F1A, F2A, F3A   # forces in reference basis A [N]
M1A, M2A, M3A   # moments in reference basis A [N m]
```

`create_Beam(..., aeroSurface=nothing)` makes UVLM the source of aerodynamic loads. Gravity is zero, the reference frame is stationary, and incidence enters through the UVLM freestream.

The mixed beam state contains displacement `u`, finite rotation parameters `p`, sectional force `F`, sectional moment `M`, linear velocity `V` and angular velocity `Omega`. AeroBeams solves the nonlinear beam equations and recovers nodal displacements and rotations.

Mesh and physical time scales

Pazy property / example setting	Value in the captured example
Modeled span / chord	0.549843728 m / 0.0989 m
Spar position from leading edge	0.44096 times the chord
Structural / aerodynamic mesh	15 elements / 10 x 20 panels
Speed / angle of attack	45 m/s / 7 degrees
`time_step_chords`	0.25

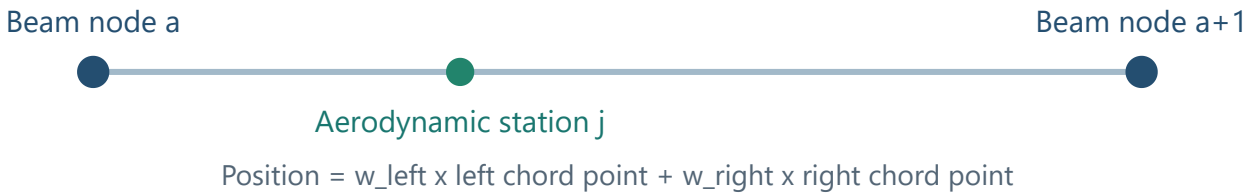
```
dt = time_step_chords * chord / airspeed
dt = 0.0005494444 s for this example
```

The dynamic calculation initializes consistent structural states, with zero initial aerodynamic nodal loads. It starts the UVLM wake from empty. It does not first call the separate coupled static solver.

Turn the deformed beam into an aerodynamic grid

`wing_geometry` builds chord lines at beam nodes, then interpolates them to aerodynamic span stations.

1. **Read each beam node.** Add recovered displacement to its reference position. Use its finite rotation and reference orientation to find the current chord direction.
2. **Place points along the chord.** The offset from the spar is `(spar_fraction - chord_fraction) * chord`, along the rotated chord direction.
3. **Convert coordinates.** Multiply by `A_TO_UVLM` to express every point in aerodynamic axes.
4. **Interpolate along the span.** Each aerodynamic station lies between two beam stations. Its row in `weights` contains their interpolation factors, which add to one.



Keep the axes explicit

UVLM direction	Corresponding AeroBeams A direction
+x: downstream	$-y_A$
+y: along the modeled span	$+z_A$
+z: upward	$-x_A$

The grid shape is `(3, nc+1, ns+1)`: three coordinates at every chordwise/spanwise vertex. The example gives `(3, 11, 21)`, independent of the 16 beam nodes.

> **Save the force moment arms too.** The returned `vortex_offsets` are offsets from each contributing beam node to its vortex-row point, in AeroBeams axes. Interior vortex rows sit a quarter panel aft of physical grid rows; the final row stays at the trailing edge.

The root is imposed exactly from the clamp when building the grid. This keeps its location on the aerodynamic symmetry plane.

UVLM solves the airflow on the guessed wing shape

At a given trial geometry, UVLM finds vortex circulation that satisfies no normal flow through the moving lifting surface. Circulation and its change in time then produce aerodynamic loads.

1. **Start from accepted history.** `evaluate_trial!` copies `state.system` into a private `workspace.system`. Each repeated trial starts from the same accepted geometry, circulation and wake.
2. **Install the guessed grid.** `apply_trial_geometry!` calls `grid_to_surface_panels` to build vortex rings, control points and normals.
3. **Calculate relative motion.** `get_surface_velocities!` uses the accepted and trial geometries. Its stored contribution is `(old_position - new_position) / dt`, which is added to fluid velocity.
4. **Assemble and solve circulation.** `propagate_system!` forms the aerodynamic influence matrix and no-penetration right-hand side. `circulation!` solves the linear system.
5. **Calculate and distribute forces.** `near_field_forces!` calculates segment and unsteady panel forces. `imperial_nodal_forces` distributes them to vortex vertices.

```
AIC * Gamma_trial = w
Gamma_dot = (Gamma_trial - Gamma_accepted) / dt
```

AIC describes how the bound vortices influence panel control points. **w** includes freestream, wing motion and accepted wake influence, projected on panel normals. The matrix is rebuilt for the changing geometry.

Where the force comes from

Contribution	Meaning in the code
Vortex-segment force	Depends on density, circulation jump, local relative velocity and segment direction.
Unsteady panel force	Depends on density, panel area, its normal and the time derivative of circulation.

Local flow includes bound- and wake-induced velocity. With symmetry enabled, image vortices influence the explicitly modeled wing. Returned loads still refer to that modeled surface.

> **A trial does not advance physical wake history.** `evaluate_trial!` uses `advance_wake=false`. It can update the trial wake attachment near the trailing edge while leaving the accepted wake at time n.

Forces are dimensional SI values. The historical `imperial_*` function names identify the force convention, not the unit system.

A vertex force can bend and twist the wing

`beam_loads` uses the same interpolation weights as the geometry transfer, in the reverse direction.

1. **Rotate the force into AeroBeams axes.** Apply the transpose of `A_TO_UVLM` to each aerodynamic vertex force.
2. **Distribute force to beam nodes.** Multiply by the station interpolation weights and add the contributions to the first three rows of `nodal_loads`.
3. **Add the moment about each node.** Use that node's saved chord offset and the cross product `offset x force`. Add the weighted moment to the last three rows.
4. **Refresh the structural boundary conditions.** `apply_pazy_nodal_loads!` writes the six numeric components into the active model and calls `update_BC_data!`.

```
force_A = A_TO_UVLM' * force_UVLM
F_node += weight * force_A
M_node += weight * cross(node_vortex_offset, force_A)
```

A small numerical example

Suppose one vertex has upward UVLM force `[0, 0, 4] N`. After axis conversion it is `[-4, 0, 0] N` in AeroBeams A. Let its beam-node weights be 0.75 and 0.25, with a chord offset of `[0, 0.02, 0] m` at each node.

Contribution	Left node, weight 0.75	Right node, weight 0.25
F1A	-3 N	-1 N
M3A	0.06 N m	0.02 N m

The total vertical force is preserved, and the force's offset generates torsion. These values illustrate the transfer; they are not simulation results.

> **Why reuse the same weights?** The geometry and load transfers then agree in virtual work: aerodynamic force acting through a small mapped displacement gives the same work as the transferred beam forces and moments.

Use each beam node's own rotated chord offset for the cross product. Using the interpolated aerodynamic point minus the beam-node position would add spanwise moment arms that do not belong to this motion interpolation.

The result is a `(6,16)` array: three forces and three moments at every beam node. Loads remain fixed within one structural Newton solve and are refreshed between coupling trials.

Advance from time n to time n+1

The coupling function owns the time loop and calls the AeroBeams and UVLM step functions in order.

1. **Select the interval.** `update_time_variables!` selects the next structural time. Update the reference orientation and the flow speed, including an optional startup ramp.
2. **Predict the next geometry.** Use `grid_guess = 2*grid - previous_grid`, with the same extrapolation for moment arms. At the first step the two saved grids are identical.
3. **Evaluate the prescribed tip pulse.** If enabled, the pulse is constant during all coupling trials at this physical time.
4. **Freeze the accepted structural history.** Call `get_equivalent_states_rates!` once to form the time-integration history terms.
5. **Open the aerodynamic interval.** Call `begin_time_step!`. Solve one loose pass, or repeat strong coupling until geometry and loads meet the tolerances.
6. **Accept and record.** Save aerodynamic nodal loads, update accepted interface geometry, call `commit_time_step!`, then save structural states and response diagnostics.

Accepted state versus trial state

Storage	What happens during repeated FSI trials
`state.system`	Retains accepted aerodynamic geometry, circulation and wake at n.
`workspace.system`	Is recreated from accepted history and solved for each guessed n+1 geometry.
Structural equivalent rates	Stay fixed, so all structural solves refer to the same previous physical state.
Structural trial states	Change as Newton solves for the current applied nodal loads.

> **Repeated trials do not mean repeated physical time.** They are different attempts to find a consistent solution at the same n+1. Rebuilding history from each trial would change the problem being solved.

The optional tip pulse is a smooth signed force in F1A. `settling_time` sets its start time; it does not automatically enforce equilibrium or remove startup data from a later damping fit.

Strong coupling repeats geometry and load exchanges

At each FSI iteration, UVLM calculates loads on the guessed shape. AeroBeams solves under those loads and returns a new shape. The interface residuals measure how much these two descriptions disagree.

```
Physical time loop
  FSI loop at the same n+1
    UVLM trial on the guessed geometry
    Transfer loads to AeroBeams boundary conditions
    Structural Newton loop: solve beam residual equations
    Recover the structural candidate geometry
    Check geometry and load residuals
    Accept, or relax geometry and repeat
  Commit UVLM and advance the wake once
```

Convergence check	What the code measures
Geometry residual	Maximum coordinate difference between structural candidate and trial grid, divided by chord.
Load residual	Largest normalized force or moment change between successive aerodynamic nodal-load arrays.

Force normalization uses the larger force magnitude from the two arrays, with a floor of 1 N. Moment normalization uses their moment magnitudes, with a floor equal to the force scale times chord. The prescribed tip pulse is excluded from this comparison.

The first load residual is infinite, so strong coupling needs at least two iterations. Both interface checks must pass. With geometry tolerance `1e-4`, this wing's maximum allowed coordinate mismatch is about 9.89 micrometres.

Relax the next guess

```
new_guess = (1 - relaxation)*old_guess
            + relaxation*structural_candidate
```

The same relaxation is applied to grid coordinates and moment arms. A value of 0.5 moves halfway toward the candidate.

> **Final consistency check.** When the two checks pass, UVLM is evaluated again on the structural candidate itself. If the resulting load change also passes, that trial is accepted. There is no additional structural solve after this final check, so agreement is controlled by the tolerances.

`:loose` accepts one UVLM/structural pass without interface convergence. FSI failure stops this fixed-dt run; aerodynamic rollback does not restart the structural step.

AeroBeams supplies the nonlinear time integration

The Pazy coupling calls AeroBeams' own step solver. The structural and interface convergence loops solve different equations.

What Newton does at fixed applied loads

1. **Assemble structural equations.** `assemble_system_arrays!` evaluates element kinematics, finite rotations, strains, momenta, equilibrium and compatibility.
2. **Include nodal loads and constraints.** Special-node equations include the clamp and the six-component UVLM load boundary conditions.
3. **Solve a linear correction.** The structural Jacobian and residual give `Delta_x = -Jacobian \ residual`. Update the structural state and repeat as needed.
4. **Recover nodal motion.** After convergence, `element_nodal_states!` produces the displacements and rotations read by `wing_geometry`.

`solve_time_step!` selects the nonlinear `solve_NewtonRaphson!` path. Its convergence accepts either an absolute residual norm or a relative state-correction norm below the selected tolerance. These tests are separate from the FSI geometry and load checks.

The time-integration relation

For an applicable state q, AeroBeams forms equivalent rates once from the accepted state, then recovers the trial rate during residual evaluation:

```
qdot_equiv = qdot_n + (2/dt)*q_n
qdot_n+1   = (2/dt)*q_n+1 - qdot_equiv
```

Rearranging gives the implicit trapezoidal relation:

```
q_n+1 = q_n + (dt/2)*(qdot_n + qdot_n+1)
```

This is applied to the first-order structural states and rates, including nonlinear rotation kinematics. For ordinary second-order dynamics, trapezoidal integration gives the familiar Newmark average-acceleration relation.

> **How your open files fit.**
> `Kinematics.jl` supplies `copy_surfaces_to_previous!` for the UVLM motion calculation. The Pazy grid mapping is its own `wing_geometry`.
> `StructuralTimeIntegration.jl` supplies separate matrix-based Newmark/generalized-alpha helpers for other examples. This Pazy runner uses AeroBeams' `Problem.jl` and `Core.jl`.

This runner does not select an independent structural damping matrix or material damping coefficient.

The wake carries aerodynamic history forward

After acceptance, `commit_time_step!` copies the final trial system and calls `advance_wake!`. Existing wake vertices move with their calculated flow velocities, and a new row is shed using trailing-edge circulation.

```
commit_time_step!
  advance_wake!
    get_wake_velocities!
    shed_wake! -> translate_wake! -> rowshift!
  commit_wake_rows!
  adopt the new system, time and step index
  save aerodynamic histories when requested
```

The wake starts empty, grows to `maximum_wake_rows`, then discards old rows as new rows enter. Increasing FSI iterations does not increase its physical age or row count.

```
Nominal retained wake length = maximum_wake_rows * U * dt
Nominal retained wake age = maximum_wake_rows * dt
```

In the captured example, 20 rows and `U*dt = 0.25*chord` give a nominal length of 5 chords. The free wake deforms, so its actual shape need not be straight.

Where to inspect the result

Result field	Meaning
`structural_problem`	AeroBeams model, final state and saved states/rates.
`aerodynamic_problem`	Accepted UVLM system, trial workspace and histories.
`time`, `dt`	Physical sample times and time step.
`tip_out_of_plane`	Tip bending displacement in metres: `-u_n2[1]` in A.
`tip_twist_degrees`	Angle from the rotated local tip chord direction.
`aerodynamic_nodal_load_history`	Beam forces/moments in A; excludes the pulse.
`coupling_iterations`	FSI exchange count at each accepted step.
`coupling_geometry_residual` `coupling_load_residual`	Final interface mismatches for each physical step.

Optional aerodynamic results store circulation and total force/moment in UVLM axes. Their saved time vector starts on accepted aerodynamic steps, while the structural history includes the initial state.

With a single surface group, **`aerodynamic_interaction`** does not switch off its own induced flow or wake development. Also, disabling GIF generation in the single-case runner does not itself disable snapshot collection; the coupling helper has a separate **`save_animation_history`** option.

Hold each shape fixed, then seek structural balance

The static runner reuses the geometry and load transfers, with a structural `SteadyProblem` and a different outer iteration.

1. **Build and solve the unloaded beam.** Use the same Pazy properties, clamp and nodal load boundary conditions.
2. **Build the current aerodynamic shape.** `wing_geometry` reads the current static structural solution.
3. **Settle a fresh wake on this fixed shape.** `settle_static_uvlm` creates a new aerodynamic problem. Repeated UVLM time steps develop the wake while the wing stays still.
4. **Check aerodynamic settlement.** Test the peak-to-peak change of force and circulation over a whole trailing window, after a minimum development time.
5. **Check static interface agreement.** Compare settled aerodynamic loads with the loads used for the current structural shape. Also check the previous shape change.
6. **Accept or update.** If the checks pass for enough consecutive iterations, stop. Otherwise relax the applied loads, solve nonlinear static beam balance, and repeat.

```
applied_loads = (1 - static_relaxation)*applied_loads
              + static_relaxation*settled_aerodynamic_loads
```

Each new outer shape starts with an empty wake. This avoids turning a static shape correction into a fictitious physical wing velocity in `(old_geometry-new_geometry)/dt`.

Question	Dynamic coupling	Static coupling
Beam equations	Transient balance, with inertia	Static nonlinear balance
Relaxed quantity	Grid and moment arms	Applied nodal loads
Wake history	Continues in physical time	Restarted for each shape
Meaning of time	Actual wing response time	Wake-development time at fixed shapes

> **Check the convergence flag.** The static result reports `converged` and `termination_reason`. A time limit, an unsettled wake or an outer iteration limit does not establish equilibrium.

The captured static runner has its own settings: 55 m/s, 2 degrees, a 4 x 15 aerodynamic mesh and 40 retained wake rows. Its cumulative aerodynamic time is not a dynamic wing-motion record. It does not initialize the current dynamic runner automatically.

Estimate whether an oscillation grows or decays

The dynamic runner can send the tip-twist history to `moving_block_damping`. The sweep uses the related `moving_block_metrics` function, which also reports availability and fit diagnostics.

1. **Select a signal segment.** The routine uses qualifying local peaks to trim the record. The Pazy runners do not automatically remove all pre-pulse data.
2. **Move a window through the signal.** Subtract each window's mean, take an FFT and record the dominant nonzero-frequency amplitude.
3. **Fit amplitude growth.** Fit a straight line to log amplitude against window time. Its slope is the growth rate, lambda.
4. **Report frequency and damping.** Use the median dominant frequency and convert growth to the reported damping ratio.

```
log(amplitude) = constant + lambda * time
omega = 2*pi*frequency
damping [%] = -100*lambda / sqrt(lambda^2 + omega^2)
```

Growth rate	Interpretation of the fitted oscillation
lambda < 0	Decaying response; positive reported damping
lambda > 0	Growing response; negative reported damping
lambda near zero	Weak growth/decay; may be unresolved

How the sweep obtains a boundary

`run_pazy_stability_sweep.jl` repeats fresh simulations over speeds and incidence angles, saves responses, fits growth, and classifies cases. The captured strict selection uses fit R-squared of at least 0.90 and growth magnitude greater than 0.05 per second.

`sweep_crossings` interpolates between adjacent finite growth rates with opposite signs. A negative-to-positive crossing is an onset; a positive-to-negative crossing is an offset. Strict results go to `uvlm_boundary.csv`. Separate bracket and provisional outputs retain less certain estimates.

> **Interpret the fit with its response.** Startup, several competing frequencies and amplitude saturation can make one exponential fit less representative of a single mode. A converged FSI step also does not establish mesh, time-step or wake-length independence.

`plot_pazy_flutter_boundaries.jl` plots boundary tuples entered in the script. It does not read the spreadsheet automatically. AeroBeams eigenvalue curves in the comparison are cached comparison results.

The sweep has its own settings. Changing the single-case runner does not change them.

File and function reference

Paths are relative to the repository root. **Pazy/** below means `lib/WingPropellerUVLM/examples/pazy_aerobeams_coupled/` **UVLM/** means `lib/WingPropellerUVLM/src/`.

File	Functions / reading purpose
`Pazy/run_pazy_wing_uvlm.jl`	Flight condition, controls and call to `run_pazy_wing_uvlm`.
`Pazy/PazyWingUVLMCoupling.jl`	`run_pazy_wing_uvlm`: construction, physical time and FSI loops.
`Pazy/PazyWingUVLMCoupling.jl`	`wing_geometry`, `beam_loads`, `apply_pazy_nodal_loads!`, `interface_load_residual`, `wingtip_twist_degrees`.
`UVLM/UVLMModel.jl` `UVLM/OperatingPoint.jl`	`create_UVLMModel`, `create_UVLMSolver`, `create_OperatingPoint`.
`UVLM/UVLMProblem.jl`	`begin_time_step!`, `evaluate_trial!`, `commit_time_step!`, `rollback_time_step!`.
`UVLM/wing_propeller/Kinematics.jl`	`copy_surfaces_to_previous!`: preserve accepted panels for motion velocities.
`UVLM/backend/geometry.jl`	`grid_to_surface_panels`: grid to vortex panels.
`UVLM/backend/analyses.jl`	`propagate_system!`, `advance_wake!`.
`UVLM/backend/circulation.jl`	`get_surface_velocities!`, `normal_velocity!`, `circulation!`.
`UVLM/backend/nearfield.jl`	`near_field_forces!`, `imperial_nodal_forces`, `imperial_nodal_positions`.
`UVLM/backend/wake.jl`	`get_wake_velocities!`, `shed_wake!`, `translate_wake!`, `rowshift!`.
`src/SampleModels.jl`	Pazy geometry, stations, sectional stiffness and inertia helpers.
`src/Beam.jl`, `BC.jl`, `Model.jl`	`create_Beam`, `create_BC`, `create_Model`.
`src/Problem.jl`	`create_DynamicProblem`, `solve_initial_dynamic!`, `get_equivalent_states_rates!`, `solve_time_step!`, `save_time_step_data!`.
`src/SystemSolver.jl`	`solve_NewtonRaphson!`, `assemble_system_arrays!`, `solve_linear_system!`.
`src/Core.jl`	`element_states_rates!`, `element_residual!`, `special_node_residual!`, `element_nodal_states!`.
`Pazy/PazyWingUVLMStatic.jl`	`run_pazy_wing_uvlm_static`, `settle_static_uvlm`.
`lib/WingPropellerUVLM/studies/src/MovingBlockDamping.jl`	`moving_block_damping`, `moving_block_metrics`.

Sources and further detail

Expanded explanation and source-line links: `docs/src/pazy-uvlm-aerobeams-walkthrough.md`

Captured source with file/function navigation and hashes: `docs/src/pazy-uvlm-aerobeams-source.html`